



INDIA.RUNS

Build what next India runs on

Team Name : babu

Team Leader Name : Meruva Peddababu

Problem Statement :

Recruiters go through hundreds of profiles and still miss the right person — not because talent isn't there, but because keyword filters can't see what actually matters. Build an AI system that ranks candidates the way a great recruiter would: understand what the role truly needs, evaluate the full picture (career history, skills, behavioral signals, platform activity), and deliver a shortlist a recruiter can trust.

Solution Overview

- Proposed Solution:
- Rule-based + semantic hybrid ranker scoring 100K candidates in <35s on CPU with zero external APIs. Four weighted dimensions: Skill Match (30%), Career Depth (35%), Behavioral Signals (20%), Location Fit (15%). Top-1,000 pool semantically re-ranked with BM25 + sentence-transformers cosine similarity.
- Differentiators from Traditional Keyword Matching:
- Career text analysis detects shipped production systems (retrieval, ranking, A/B testing) rather than listed buzzwords. Availability is a multiplicative signal — inactive high-skill candidates rank below active lower-skill ones. Five honeypot detectors catch impossible/fake profiles before scoring. Semantic re-ranking on top-1K only — precision boost without embedding all 100K candidates.

JD Understanding & Candidate Evaluation

- Key Requirements Extracted from JD:
- Must-have: embeddings, vector DB, ranking systems, Python. Bonus: LangChain, LLM fine-tuning, MLOps. YoE band: 4–9 years (soft — production evidence outweighs raw years). Location: Pune or Noida preferred. Seniority signals: shipped A/B experiments, NDCG/MRR evaluation, real-time inference, production deployment.
- Evaluating Candidate Fit Beyond Keyword Matching:
- Career descriptions scanned for 4 evidence categories: `production_retrieval` (35%), `production_ranking` (35%), `production_ml` (20%), `eval_framework` (10%). A candidate who wrote 'built hybrid search over 50M profiles' scores higher than one who listed every buzzword. Behavioral signals (last active date, response rate, open-to-work flag) act as a multiplier — not additive — so inactive candidates never top the list.

Ranking Methodology

- Retrieve → Score → Rank:
- Stream candidates.jsonl → honeypot check → 4-dimension composite × availability multiplier → top-1,000 max-heap → BM25 + sentence-transformers re-rank → top-100 CSV.
- Models / Algorithms / Heuristics:
- Skills: JD taxonomy (must-have ×1.0, bonus ×0.4) + endorsement×duration trust multiplier. Career: regex against 4 evidence categories. Behavioral: day-decay on last-active + open-to-work boost. Location: tiered multiplier (1.0/0.9/0.78/0.55). Re-rank: BM25 + all-MiniLM-L6-v2 blend (0.4/0.6).
- Combining Multiple Signals:
- $$\text{final_score} = (\text{skill} \times 0.30 + \text{career} \times 0.35 + \text{behavioral} \times 0.20 + \text{location} \times 0.15) \times (0.45 + 0.55 \times \text{availability})$$

Multiplicative availability ensures unreachable candidates never dominate regardless of skill score.

End-to-End Workflow

- 1 Parse JD (text/DOCX) — extract must-have skills, bonus skills, YoE band, location, seniority signals.
- 2 Stream candidates.jsonl — one record at a time; never loads full dataset into memory.
- 3 Per candidate: run 5 honeypot checks → if clean, compute 4-dimension score × availability multiplier.
- 4 Maintain top-1,000 max-heap; concurrent.futures parallel workers for throughput.
- 5 Semantic re-rank: BM25 index + sentence-transformer embeddings for pool; blend (×0.4 / ×0.6).
- 6 Emit top-100 → submission.csv: candidate_id, rank, score, reasoning.
- 7 Validate: validate_submission.py — 0 errors, monotone scores, 0 honeypots in top 100.

System Architecture

Input Layer: JD (text/DOCX) + candidates.jsonl (100K records, streaming)

Honeypot Filter: 5 rule-based detectors — any trigger sets score = 0.0 before scoring

4-Dimension Scorer (concurrent.futures): Skill Match | Career Depth | Behavioral | Location

Top-1K Heap: max-heap maintains top 1,000 candidates without full-dataset sort

Semantic Re-ranker: BM25 (rank-bm25) + all-MiniLM-L6-v2 cosine blend (CPU, no GPU)

Output: submission.csv — candidate_id, rank, score, reasoning (top 100, monotone)

Optional UI: Streamlit app — live score breakdown, filtering, plotly visualisations

Runtime: ~35s / 100K candidates on M2 Pro CPU only | <400MB RAM

Results & Performance

- Top 5 Ranked Candidates (all India-based ML/Search engineers with production evidence):
- #1 CAND_0018499 0.878 — 7yr Sr. ML Eng @ Zomato, embeddings+weaviate, Noida, 15d notice
- #2 CAND_0046525 0.874 — 6yr Sr. ML Eng @ Genpact AI, embeddings+elasticsearch, Pune
- #3 CAND_0081846 0.869 — 7yr Lead AI Eng @ Razorpay, embeddings+pgvector, Jaipur
- #4 CAND_0055905 0.865 — 8yr Sr. ML Eng @ Flipkart, embeddings+opensearch, 30d notice
- #5 CAND_0002025 0.848 — 6yr Sr. AI Eng @ Apple, embeddings+opensearch, Trivandrum
- 0 honeypots in top 100. Passes validate_submission.py with 0 errors.
- Runtime & Compute:
- <35s for 100K candidates on M2 Pro (CPU only, no GPU). <400MB RAM. Zero external APIs. Python 3.9–3.12.

Technologies Used

- Python 3.11 stdlib — zero-dependency core ranker; cross-platform
- sentence-transformers (all-MiniLM-L6-v2) — CPU-friendly semantic embeddings for re-ranking
- rank-bm25 — fast lexical scoring for top-1K pool
- concurrent.futures — parallel candidate scoring, no external deps
- NumPy / scikit-learn — numerical ops, no GPU required
- Streamlit + pandas + plotly — interactive UI (optional; not in core ranking path)
- AI Tools used: ChatGPT for code review and debugging Python issues. No candidate data fed to any LLM.

Submission Assets

GitHub Repo: <https://github.com/MeruvaPeddababu/redrob-ranker>

Demo / Sandbox: <https://redrob-ranker-jg4ke7xwphuzny8aexo6jx.streamlit.app/>

Run command: `python3 ranker_v3.py --jd job_description.docx --candidates candidates_simplified.json --output results.csv"`



INDIA.RUNS

Build what next India runs on

THANK YOU